ChocoPy v2.2:
Руководство и справочник по языку

Разработано Роханом Падхье и Коушиком Сеном; изменения версии v2 — Полом Хилфингером. Калифорнийский университет.Беркли. 23 ноября 2019 г.

Адаптировано на русский язык — Кулагин Иван Иванович. ИСП РАН. 2026 г.

1. Введение 1
2. Обзор Chocopy 2
   2.1. Структура программы 3
   2.2. Функции 4
   2.3. Классы 5
   2.4. Иерархия типов 7
   2.5. Значения 8
   2.6. Выражения 10
   2.7. Аннотации типов 13
   2.8. Операторы 13
3. Лексическая структура 17
   3.1. Структура строки 17
   3.2. Идентификаторы 19
   3.3. Ключевые слова 19
   3.4. Литералы 19
   3.5. Операторы и разделители 21
4. Грамматика языка 21
   4.1. Приоритет и ассоциативность операторов 21
5. Правила проверки типов 24
   5.1. Среды типов 24
   5.2. Правила проверки типов 26

1. Введение

В настоящем руководстве описывается язык ChocoPy, который является статически типизированным диалектом Python 3.6. ChocoPy предназначен для использования в учебных целях. Он был разработан для того, чтобы студенты могли реализовать полный компилятор ChocoPy за один семестр. ChocoPy является подмножеством Python. Почти каждая допустимая программа ChocoPy также является допустимой программой Python 3.6. Выполнение программы ChocoPy, которое не приводит к ошибке, обычно имеет ту же наблюдаемую семантику, что и выполнение этой программы в Python 3.6. В Приложении A перечислено небольшое количество исключений из этого правила. Программа ChocoPy должна находиться в одном исходном файле. На верхнем уровне программа ChocoPy состоит из последовательности определений переменных, определений функций и определений классов, за которыми следует последовательность операторов. Класс состоит из последовательности определений атрибутов и определений методов. Класс создает определяемый пользователем тип. Определения функций могут быть вложены в другие методы и функции. Все имена классов и функции, определенные на верхнем уровне, видны глобально. Классы, функции и методы не могут быть переопределены. Программные операторы могут содержать выражения, присваивания и операторы потока управления, такие как условные операторы и циклы. Вычисление выражения формирует значение, которое может быть целым числом, логическим значением, строкой, объектом пользовательского класса, списком или специальным значением `None`. ChocoPy не поддерживает словари, функции как объекты первого класса и рефлексивную интроспекцию.

Все выражения статически типизированы. Переменные (глобальные и локальные) и атрибуты классов статически типизированы и имеют только один тип на протяжении всего своего жизненного цикла. Атрибуты и переменные явно типизированы с помощью аннотаций. В определениях функций и методов аннотации типов используются для явного указания типа возвращаемого значения и типов формальных параметров. Для читателей, знакомых с языком Python, на рисунке 1 приведен пример кода на языке ChocoPy, иллюстрирующий общую структуру программы, которая содержит глобальные функции, операторы, глобальные переменные, локальные переменные и аннотации типов. Аннотации типов являются корректными синтаксическими конструкциями в Python 3.6, однако интерпретатор Python просто игнорирует эти аннотации и оставляет их как подсказки для других инструментов, например, линтеров (linters). В отличие от интерпретатора Python ChocoPy применяет статическую проверку типов во время компиляции.

На рисунке 1 функция `is_zero` определена на верхнем уровне программы. Ее формальные параметры `items` и `idx` явно типизированы как список целых чисел и целое число соответственно. Возвращаемый тип функции `is_zero` — `bool`. Функция определяет локальную переменную `val`, тип которой — `int`. На верхнем уровне программа определяет глобальную переменную `mylist`, тип которой — список целых чисел. Функция `is_zero` вызывается в операторе верхнего уровня, а ее результат выводится с помощью предопределенной функции `print`. Аналогично, на рисунке 2 показана программа ChocoPy, которая определяет два класса: `animal` и `cow`. Класс `cow` наследуется от `animal`, который, в свою очередь, наследуется от предопределенного корневого класса `object`. Атрибут `makes_noise` типа `bool` определен в `animal` и, следовательно, наследуется классом `cow`. Класс `cow` переопределяет метод `sound`. Конструктор для класса `cow` вызывается в строке 19. Раздел 2 содержит подробный, но неформальный обзор различных языковых конструкций в ChocoPy. Разделы 3–5 содержат формальные описания лексической структуры, грамматического синтаксиса, правил типизации и семантики операций ChocoPy.

2. Обзор Chocopy

Обозначения. В настоящей спецификации используются следующие обозначения:

- `{expr}` — для обозначения выражения в программе;
- `{id}` — для обозначения идентификатора, такого как имя переменной или функции;
- `{stmts}` — для обозначения списка операторов программы, разделенных символами новой строки;
- `{declarations}` — для обозначения списка (возможно, чередующихся) объявлений функций, переменных, атрибутов и/или классов, где это применимо;
- `{type}` — для обозначения аннотации статического типа;
- `{literal}` — для обозначения константного литерала, такого как целочисленный литерал, строковый литерал или ключевые слова `True`, `False` или `None`.

```
1 def is_zero (items : [int ], idx : int ) -> bool:
2   val:int = 0            # Type is explicitly declared
3   val = items [idx]
4   return val == 0
5
```

```
6  mylist:[int] = None
7  mylist = [1, 0, 1]
8  print(is_zero(mylist, 1)) # Prints 'True'
```

Рисунок 1: Программа ChocoPy, иллюстрирующая функции, переменные и статическую типизацию

2.1. Структура программы

Программа ChocoPy состоит из нуля или более определений, за которыми следуют ноль или более операторов, называемых глобальными определениями и глобальными операторами соответственно. Глобальные определения включают определения глобальных переменных, определения функций и определения классов. Эти определения формируют *глобальную область видимости* (*global scope*). Глобальные переменные определяются с использованием синтаксиса `{id}:{type} = {literal}`, где *идентификатор* (`id`) указывает имя переменной, аннотация типа указывает статический тип переменной, а константный литерал указывает начальное значение переменной при выполнении программы. Имена глобальных переменных, глобальных функций и классов должны быть уникальными. Глобальные операторы выполняются в контексте глобальной области видимости; то есть выражения в глобальных операторах могут ссылаться на сущности, определенные в глобальной области с помощью идентификаторов. Выполнение программы ChocoPy начинается с первого глобального оператора и заканчивается, когда последний глобальный оператор будет выполнен полностью. Определения функций и классов описаны в Разделе 2.2 и Разделе 2.3 соответственно. Операторы программы описаны в Разделе 2.8.

2.2. Функции

В ChocoPy определение функции может появляться на верхнем уровне программы или может быть вложено в другие функции или методы. Функции не могут быть переопределены в той же области видимости. Однако определение функции в текущей области видимости может перекрыть (затенить, shadow) функцию, определенную в области видимости текущей функции. Определение функции имеет следующую форму:

```
def {id}({id}: {type}, ... , {id}: {type}) {return type}:
  {declarations}
  {stmts}
```

где `{return type}` либо пуст, либо имеет вид `-> {type}`.

Первая строка определяет имя функции, в скобках — список из нуля или более *формальных параметров*, разделенных запятыми, и возвращаемый тип функции после символа `->`. Если возвращаемый тип пуст, функция может возвращать только значение `None`. Каждый формальный параметр имеет имя и аннотацию статического типа. Тело функции содержит последовательность из нуля или более объявлений, за которой следует последовательность из одного или более операторов программы. Определение функции создает новую *область видимости*. Объявления в теле функции включают определения локальных переменных, объявления глобальных и нелокальных переменных и определения вложенных функций. `{id}:{type} = literal}`. Такое определение объявляет локальную переменную с именем `id`, явно связывает ее со статическим типом и указывает начальное значение с помощью литерала. Оператор `global {id}` используется для привязки имени к глобальной переменной. Аналогично, оператор `nonlocal {id}` используется внутри вложенной функции для привязки имени к переменной, определенной в области, которая не является *глобальной областью видимости*, а является ближайшей окружающей для области видимости, в которой объявлена эта переменная. Недопустимо, чтобы глобальное объявление (`global`) использовалось в глобальной области видимости. Аналогично, нелокальное объявление (`nonlocal`) недопустимо использовать вовне вложенной функции или для ссылки на глобальную переменную. Если переменная явно не объявлена в функции, но связана с некоторой сущностью — переменной, функцией или классом — в любой окружающей области видимости, то ее связывание неявно наследуется из окружающей области видимости как переменная только для чтения. Такая переменная не может быть использована в качестве целевой в операциях присваивания ни в одном из операторов функции.

```
1   class animal(object) :
2     makes_noise:bool = False
3
4     def make_noise(self:"animal") -> object:
5       if(self.makes_noise):
6         print(self.sound ())
7
8     def sound(self:"animal") -> str:
9       return "??? "
10
11  class cow(animal):
12    def __init__(self:"cow "):
```

```
13      self.makes_noise = True
14
15    def sound(self:"cow ") -> str:
16      return "moo "
17
18  c:animal = None
19  c = cow ()
20  c.make_noise () # Prints "moo"
```

Рисунок 2: Программа ChocoPy, иллюстрирующая классы, атрибуты, методы и наследование

2.3. Классы

В ChocoPy определение класса может появляться на верхнем уровне программы. Классы не могут быть переопределены. Имена классов никогда не могут использоваться для именования любых переменных или функций. Определение класса имеет следующую форму:

```
class {id}({id}):
  {declarations}
```

В первой строке указывается имя класса, за которым в скобках следует имя его *суперкласса*. Имя класса не должно быть связано ни с какой другой сущностью — классом, функцией или переменной — в программе. *Суперкласс* должен ссылаться на класс, который был ранее определен в программе, или на предопределенный класс `object`. Суперкласс не может быть специальным предопределенным классом `int`, `str` или `bool`.

Тело класса состоит из последовательности определений атрибутов и определений методов. В ChocoPy атрибуты и методы связаны с экземплярами объектов класса, а не с самими классами; то есть ChocoPy не поддерживает понятие статических членов класса, которое поддерживают некоторые другие языки. Подобно определениям переменных, определение атрибута имеет форму `{id}:{type} = {literal}`. Определение метода имеет тот же синтаксис, что и определение функции (см. Раздел 2.2), с двумя важными ограничениями: (1) определение метода должно иметь, по крайней мере, один формальный параметр, и (2) первый формальный параметр должен иметь тип определяющего класса.

Класс определяет атрибуты и методы. Класс наследует атрибуты и методы своего суперкласса. Атрибуты, определенные в текущем классе или унаследованные от суперкласса, не могут быть переопределены. Методы не

могут быть переопределены в том же классе. Унаследованные методы могут быть переопределены, пока возвращаемый тип и типы всех формальных параметров, за исключением первого параметра, являются точно такими же. Любая ссылка на атрибут или метод должна иметь префикс в виде выражения и оператора точки (см. Раздел 2.6).

Если класс `C` определен как имеющий суперкласс `P`, то класс `C` является подклассом `P`. Если `C` является подклассом `P`, то `P` должен быть либо определенным пользователем классом, который был определен до `C` в программе, либо быть предопределенным классом `object`. Класс `object` не имеет суперкласса. Поскольку каждый класс ChocoPy (кроме `object`) наследует атрибуты и методы от одного суперкласса, эта схема называется *одиночным наследованием*. Отношение подкласс/суперкласс в классах определяет граф. Поскольку класс может быть подклассом только другого класса, определенного ранее, этот граф представляет собой дерево с `object` в качестве корня.

Чтобы создать объект `o` типа `C`, используется выражение вызова конструктора `C()`. При выполнении сначала создается новый объект с атрибутами, инициализированными их определенными значениями. Затем вызывается метод `__init__` в классе `C`. Если метод `__init__` не определен в классе `C`, то вызывается унаследованный метод `__init__`. Корневой класс `object` имеет метод `__init__` по умолчанию, тело которого пусто.

2.4. Иерархия типов

В ChocoPy каждое имя класса также является типом. Основное правило типов в ChocoPy заключается в том, что если метод или переменная ожидает значение типа `P`, то вместо него может быть использовано любое значение типа `C`, при условии, что `P` является предком `C` в иерархии классов. Другими словами, если `C` наследуется от `P`, напрямую или косвенно, то `C` может использоваться везде, где `P` было бы достаточно. Когда объект типа `C` может использоваться вместо объекта типа `P`, мы говорим, что `C` *соответствует* `P` или что `C ≤ P` (интерпретируется как `C` находится ниже в дереве наследования чем `P`). *Соответствие типов* классов определяется в терминах графа наследования. Пусть `A`, `C` и `P` будут типами. Тогда соответствие (т. е. отношение ≤) определяется следующим образом:

- `A ≤ A` для всех типов `A`
- если `C` является подклассом `P`, то `C ≤ P`

- если `A ≤ C` и `C ≤ P`, то `A ≤ P`

Корнем иерархии классов является предопределенный класс `object`. Предопределенные типы `int`, `bool` и `str` являются подклассами `object`. Кроме того, для каждого типа `T` в программе ChocoPy существует тип списка `[T]`, который представляет список, элементы которого имеют тип `T`. Например, тип `[int]` представляет список целых чисел. Типы списков являются рекурсивными: тип `[[int]]` представляет список, каждый элемент которого является списком целых чисел. Типы списков не связаны друг с другом отношением `≤`. Каждый тип списка соответствует `object`; то есть, `[T] ≤ object` для любого типа `T`.

В дополнение к типам, которые можно обозначить в программах ChocoPy, есть два специальных типа: тип `None`, который мы обозначаем `<None>`, и тип `[]`, который мы обозначаем как `<Empty>`. Поскольку в ChocoPy нет способа явно записать эти имена типов, никакая переменная, параметр или возвращаемое значение функции никогда не имеет ни одного из этих типов. В иерархии типов `<None> ≤ object, <Empty> ≤ object` и, как обычно, `<None> ≤ <None>` и `<Empty> ≤ <Empty>`, но в остальном эти типы не связаны ни с каким другим типом. Для описания присваиваний и вызова функций нам понадобится немного другое отношение, которое мы назовем *совместимостью присваивания* и обозначим символом $\leq_a$. Грубо говоря, идея заключается в том, что мы можем присваивать или передавать значение типа T_1 чему-либо типа T_2, если $T_1 \leq_a T_2$. Точнее, $T_1 \leq_a T_2$ тогда и только тогда, когда хотя бы одно из следующих условий верно:

- $T_1 \leq T_2$ (т. е. обычное подтипирование);
- T_1 является типом `<None>`, а T_2 не является ни одним из типов `int`, `bool` или `str`;
- T_2 является списком `[T]`, а T_1 является `<Empty>`;
- T_2 является списком `[T]`, а T_1 — `[<None>]`, где `<None>` $\leq_a$ `T`.

Последний случай заслуживает упоминания: это единственный случай, когда два разных типа списков совместимы по присваиванию. Он удобен для записи таких случаев, как

```
x: [A] = [None, None]
```

Однако, он довольно ограничен. Например, присваивание

```
x: [[A]] = [[None]]
```

все еще недопустимо, несмотря на то, что выглядит совершенно корректным. Необходимо ограничить места, где можно назначать объекты типа `[<None>]`, как описано в правиле MULTIASSIGN-SMTM в Разделе 5.

В некоторых ситуациях нам также понадобится использовать концепцию *объединения* двух или более типов. Объединение двух типов `A` и `B` — это наименьший тип `C` (использующий упорядочение $\leq_a$), такой что `A` и `B` совместимы по присваиванию с `C`. Оператор объединения ⊔ можно формально определить следующим образом: `C = A ⊔ B` тогда и только тогда, когда:

$$(A \leq_a C) \wedge (B \leq_a C) \wedge (\forall D : (A \leq_a D) \wedge (B \leq_a D) \Rightarrow (C \leq_a D))$$

То есть, `C` является объединением `A` и `B` тогда и только тогда, когда и `A`, и `B` совместимы по присваиванию с `C`, и если существует тип `D` такой, что `A` и `B` совместимы по присваиванию с `D`, то `C` также совместим по присваиванию с `D`. Объединение любых двух типов всегда существует и является уникальным:

- если $A \leq_a B$, то `A ⊔ B = B ⊔ A = B`;
- в противном случае, `A ⊔ B` — это просто наименьший общий предок `A` и `B` в древовидной иерархии типов Определено как ≤.

2.5. Значения

В ChocoPy существуют следующие виды значений.

2.5.1. Целые числа

Целые числа являются знаковыми и представлены с использованием 32 бит. Диапазон целых чисел составляет от -2^{31} до ($2^{31} - 1$). Хотя целые числа соответствуют типу `object`, они неизменяемы. Арифметические операции, которые вызывают переполнение, приводят к неопределенному поведению при выполнении программы.

2.5.2. Логические значения

Существует ровно два логических значения: `True` и `False`.

2.5.3. Строки

Строки — это неизменяемые последовательности символов. Строковые литералы разделяются двойными кавычками, например, `"Hello World"`.

Строки поддерживают следующие три операции: получение длины с помощью функции `len`, индексирование с помощью синтаксиса `s[i]` и конкатенацию с помощью синтаксиса `s1 + s2`. Как и в Python, в ChocoPy нет

символьного типа. Индексация над строкой возвращает новую строку длиной 1. Конкатенация возвращает новую строку с длиной, равной сумме длин ее операндов.

2.5.4. Списки

Списки — это изменяемые последовательности с фиксированной длиной. Таким образом, списки в ChocoPy ведут себя больше как массивы в C. Список можно создать с помощью нотации квадратных скобок, например, `[1, 2, 3]`.

Как и строки, списки типа `[T]` поддерживают три операции: `len`, индексацию через синтаксис `lst[i]` и конкатенацию через синтаксис `lst1 + lst2`. Индексация списка типа `[T]` возвращает значение типа `T`. Конкатенация двух списков типов `[T1]` и `[T2]` соответственно возвращает новый список типа `[T3]`, где тип элемента нового списка — `T3 = T1 ⊔ T2`. Длина объединенного списка равна сумме длин двух операндов. Кроме того, списки типа `[T]` изменяемые и поддерживают четвертую операцию — присваивания значения элементам с помощью синтаксиса `lst[i] = {expr}`, где выражение в правой части должно соответствовать типу `T`.

Правила ChocoPy разрешают создание некоторых довольно странных (но безвредных) списков с такими типами, как `[<None>]` и `[<Empty>]`. Единственные переменные и параметры, которым они могут быть назначены или переданы, имеют объявленный тип `object`, поскольку в ChocoPy невозможно записать конкретные имена типов `[<None>]` и `[<Empty>]`. Возможно даже (хотя и не особенно полезно) индексировать такие значения.

2.5.5. Объекты пользовательских классов

Объекты обрабатываются с помощью ссылок. То есть, `x = cow()` подразумевает, что переменная `x` ссылается на объект типа `cow`. Последующее присваивание `y = x` подразумевает, что `x` и `y` ссылаются на один и тот же объект `cow` в памяти. Оператор `is` можно использовать для определения того, ссылаются ли два выражения на один и тот же объект в памяти. Объекты уничтожаются, когда они недоступны из любой локальной, глобальной или временной переменной.

2.5.6. None

`None` — это особое значение, которое можно присвоить переменной или атрибуту типа `object`, любому определенному пользователем типу класса или любому типу списка. Оператор `is` можно использовать для определения

того, вычисляется ли выражение в значение `None`. Для проверки типов оно имеет тип `<None>` (см. раздел 2.4).

2.5.7. Пустой список ([])

Выражение `[]` создает список, который можно присвоить переменной, имеющей тип `object` или любой тип списка. Для проверки типов оно имеет тип `<Empty>`.

2.6.Выражения

ChocoPy поддерживает следующие категории выражений: литералы, идентификаторы, арифметические выражения, логические выражения, выражения сравнения (реляционные выражения), выражения конкатенации, выражения доступа и выражения вызова.

2.6.1. Литералы и идентификаторы

Базовое выражение — это константный литерал или переменная. Литералы типа `str`, `bool` и `int` были кратко описаны в Разделе 2.5, а их лексическая структура описана в Разделе 3.4. Переменные вычисляются в соответствии со значением, содержащимся в переменной. Если идентификатор привязан к глобальной функции или классу, то он не является допустимым выражением сам по себе — он может появляться только в определенных выражениях, таких как выражения вызова. Это связано с тем, что ChocoPy не поддерживает функции и классы как объекты первого класса.

2.6.2. Списки выражений

Списки могут быть созданы с использованием последовательности выражений, разделенных запятыми и заключенных в квадратные скобки: `[{expr}, ... ]`. Тип выражения списка, содержащего один или несколько элементов, — `[T]`, где `T` — наименьший общий предок типов элементов списка в иерархии типов программы. Другими словами, `T` — наименьший тип, такой, что тип каждого выражения элемента соответствует `T`. Используя оператор объединения ⊔, определенный в Разделе 2.4, мы можем сказать, что выражение вида `[{expr_1}, {expr_2}, ... , {expr_n}]`, где каждое выражение `{expr_i}` имеет тип `T_i`, приводит к списку типа `[T]`, где `T = T_1 ⊔ T_2 ⊔ ... ⊔ T_n`.

Выражение пустого списка `[]` имеет специальный тип `<Empty>`, который позволяет присваивать его (передавать как) переменной (параметр) любого типа списка. Например, если переменная x имеет тип `[int]`, то присваивание `x = []` является допустимым; x будет содержать пустой список целых чисел после этого присваивания.

2.6.3. Арифметические выражения

ChocoPy поддерживает следующие арифметические выражения с двумя операндами типа `int: {expr} + {expr}, {expr} - {expr}, {expr} * {expr}, {expr} // {expr}` и `{expr} % {expr}`. Эти операторы выполняют целочисленное сложение, вычитание, умножение, деление и вычисление остатка от деления.

ChocoPy не поддерживает выражение `{expr} / {expr}`, которое в Python вычисляется как значение с плавающей точкой. Унарное выражение `-{expr}` вычисляется как отрицательное значение целочисленного операнда. Арифметические операции возвращают значение типа `int`.

2.6.4. Логические выражения

ChocoPy поддерживает следующие логические операции над операндами типа `bool: not {expr}`, `{expr} and {expr}` и `{expr} or {expr}`, которые вычисляются как логическое отрицание, конъюнкция и дизъюнкция их операндов соответственно. Логические выражения возвращают значение типа `bool`. Если левый операнд выражения `and` вычисляется как `False`, то возвращается результат `False` без вычисления правого операнда вообще. Аналогично, если левый операнд выражения `or` вычисляется как `True`, то возвращается результат `True` без вычисления правого операнда вообще. Эта семантика важна, когда выражения в правых операндах содержат побочные эффекты.

2.6.5. Выражения сравнения

ChocoPy поддерживает следующие выражения сравнения для операндов типа `int`: `{expr} < {expr}`, `{expr} ≤ {expr}`, `{expr} > {expr}`, `{expr} ≥ {expr}`. Кроме того, операнды в выражениях вида `{expr} == {expr}` и `{expr} ≠ {expr}` может иметь тип `int`, `bool` или `str`, если оба операнда имеют один и тот же тип. Напротив, операнды в выражениях формы `{expr} is {expr}` могут быть литералом `None` или выражениями любого статического типа, отличного от `int`, `bool`, `str`. Операторы `==` и `≠` возвращают `True` тогда и только тогда, когда их операнды вычисляются как равные или неравные значения целых чисел, логических значений или строк соответственно. Оператор `is` возвращает `True` тогда и только тогда, когда оба операнда вычисляются как один и тот же объект или если оба операнда вычисляются как `None`.

2.6.6. Условные выражения (Тернарные)

Выражение `{expr1} if {expr0} else {expr2}` сначала вычисляет `{expr0}`, которое должно иметь тип `bool`. Если результат равен `True`, то вычисляется `{expr1}`, и его результат является значением выражения. В противном случае вычисляется `{expr2}`, и его значение является значением выражения.

2.6.7. Выражения конкатенации

Выражение `{expr} + {expr}` может использоваться для конкатенации двух строк или двух списков; результатом является новая строка или список соответственно.

2.6.8. Выражения доступа

К атрибуту объекта можно получить доступ с помощью оператора точки: `{expr}.{id}`. Например, `x.y.z` возвращает значение, хранящееся в атрибуте `z` объекта, полученного путем вычисления выражения `x.y`. К элементу строки или списка можно получить доступ с помощью оператора индекса: `{expr}[{expr}]`. Например, `"Hello"[2+2]` возвращает строку `"o"`. Доступ к строке или списку `x` с индексом `i` таким, что `i < 0` или `i ≥ len(x)` прерывает программу с соответствующим сообщением об ошибке.

2.6.9. Выражения вызова

Выражение вызова имеет вид `{id}({expr}, ... )`, где `{expr}, ...` — это разделенный запятыми список из нуля или более выражений, предоставленных в качестве аргументов для вызова. Если идентификатор привязан к глобально объявленной функции, выражение вычисляется в результате вызова функции. Если идентификатор привязан к классу, выражение приводит к созданию нового объекта этого класса, метод `__init__` которого вызывается с предоставленными аргументами.

Выражение вида `{expr}.{id}({expr}, ... )` вызывает метод с именем `{id}` для объекта, возвращаемого путем вычисления выражения слева от оператора точки. Первый аргумент неявный и является объектом, метод которого вызывается; остальные аргументы явно указаны в скобках. Методы вызываются с помощью динамической диспетчеризации: если динамический тип объекта, т. е. тип на момент выполнения, равен `T`, то вызывается метод `{id}`, определенный в `T` или унаследованный от `T`.

2.7. Аннотации типов

В ChocoPy аннотации статических типов используются для явного указания типов для переменных, атрибутов, формальных параметров и типов возвращаемых значений функций и методов. Аннотация типа может ссылаться либо на тип класса `T`, либо на тип списка `[T]`, такой что `T` является аннотацией типа, соответствующей типу элементов списка. Аннотации типов классов могут быть предоставлены в одной из двух форм: как идентификаторы или как строковые литералы, содержащие имя класса. В ChocoPy можно использовать любую из двух форм для аннотаций. Однако в Python первая форма не может использоваться для ссылки на тип класса, который еще не определен, поскольку Python интерпретируется построчно. Поскольку мы хотим, чтобы ChocoPy вел себя так же, как Python, мы будем использовать последнюю форму аннотации в описанном выше сценарии. В частности, строковые литералы всегда необходимы в аннотациях типов для первого формального параметра в определениях методов, поскольку тип этого параметра всегда совпадает с типом включающего класса, который еще не полностью определен.

2.8. Операторы

2.8.1. Операторы выражений

Самый простой оператор — это отдельное выражение. Выражение вычисляется, а его результат отбрасывается. Эти типы операторов полезны, когда у них есть побочные эффекты, например, `print("Hello")`.

2.8.2. Составные операторы (Compound Statements): условные операторы и циклы

ChocoPy поддерживает синтаксис `if-elif-else` в стиле Python для условного потока управления, при этом elif и else являются необязательными:

Следующий код:

```
if {expr1}:
  {body1}
elif {expr2}:
  {body2}
elif {expr3}:
  {body3}
```

эквивалентен коду:

```
if {expr1}:
  {body1}
```

```
else:
  if {expr2}:
    {body2}
  else:
    if {expr3}:
      {body3}
```

Выражения в условиях `if` и `elif` должны иметь тип `bool`. Тело, следующее сразу за условием `if` или `elif`, вычисляется только в том случае, если выражение вычисляется как `True`. Если выражение вычисляется как `False`, то рассматриваются последующие блоки `elif` или `else`. Тело, следующее за ветвью `else`, вычисляется только в том случае, если все предыдущие выражения условия вычисляются как `False`.

ChocoPy поддерживает два типа циклов: простые циклы `while` и циклы `for` по спискам и строкам. Циклы `while` имеют следующую структуру:

```
while {expr}:
  {body}
```

Выражение должно иметь тип `bool`. Тело многократно вычисляется до тех пор, пока выражение вычисляется как `True` между итерациями. Циклы `for` можно использовать для итерации по элементам списка или символам строки. Они имеют следующий вид:

```
for x in {expr}:
  {body}
```

Циклы `for` — это синтаксический сахар; приведенная выше структура эквивалентна следующему циклю типа `while`:

```
itr = {expr}
idx = 0
while idx < len(itr):
  x = itr[idx]
  {body}
  idx = idx + 1
```

где `len` — предопределенная функция определения длины, а `itr/idx` — временные переменные, которые не определены в исходной области действия. Цикл `for` не создает новых объявлений для переменной цикла (x в приведенном выше примере); переменная цикла должна быть объявлена до оператора `for`.

2.8.3. Операторы присваивания

Оператор присваивания может иметь одну из следующих трех форм:

(1) `{id} = {expr}`

присваивает значение переменной, связанной с идентификатором {id},

(2) `{expr}.{id} = {expr}`

присваивает значение атрибуту объекта, и

(3) `{expr}[{expr}] = {expr}`

присваивает значение элементу списка. При присваивании значения индексу `i` списка `x`, если `i < 0` или `i ≥ len(x)`, программа аварийно завершается после вывода соответствующего сообщения об ошибке.

Одно присваивание может присваивать одно и то же значение нескольким различным назначениям. Например, код `x = y.f = z[0] = 1` присваивает целочисленное значение `1` трем ячейкам памяти в указанном порядке: (1) переменной `x`, (2) атрибуту `f` объекта, на который ссылается переменная `y`, и (3) нулевому элементу списка `z`. То есть, сначала вычисляется конечное выражение (правая часть). Затем результат присваивается левым частям (слева от символов =), вычисляя их слева направо.

2.8.4. Оператор Pass

Оператор `pass` является пустым оператором (аналог noop). Состояние программы не изменяется, и поток управления просто переходит к следующему оператору.

2.8.5. Оператор Return

Оператор `return` завершает выполнение функции и, при необходимости, возвращает значение, используя синтаксис `return {expr}`. Если возвращаемое значение не указано, возвращается значение `None`. Недопустимо, чтобы оператор `return` находился на верхнем уровне программы вне тела функции или метода. Во время выполнения функции, если поток управления достигает конца тела функции, не встретив оператора `return`, то неявно возвращается значение `None`. Рассмотрим следующий пример:

```
def bar(x: int) -> object:
  if x > 0:
    return
```

```
elif x == 0:
  return None
else:
  pass
```

Выполнение функции `bar` может завершиться либо потому, что (1) `x > 0` и выполняется оператор `return` без возвращаемого значения, либо (2) `x == 0` и выполняется явный `return None`, либо (3) `x < 0` и поток управления достигает конца функции, неявно возвращая `None`. В функциях или методах, которые объявляют возвращаемый тип `int`, `str` или `bool`, все пути выполнения должны содержать оператор `return` с выражением, которое не является литералом `None`. В определениях классов методы `__init__` должны иметь пустой тип возвращаемого значения (указывающий, что они всегда возвращают `None`).

2.8.6. Предопределенные классы и функции

Функции `print`, `input` и `len` предоставляются средой выполнения.

`print` принимает аргумент типа `object` и выводит его в стандартный поток вывода, возвращая значение `None`. Допустимые типы аргументов ограничены: `str`, `int` или `bool`. Другие аргументы приводят к аварийному завершению программы с сообщением об ошибке.

`input` не принимает аргументов и возвращает значение типа `str`, считывая строку ввода из стандартного потока ввода, включая последний символ новой строки. Функция возвращает пустую строку, когда стандартный поток ввода исчерпан.

`len` принимает аргумент x типа `object`, возвращая его длину, если это `str` или `list`. Другие аргументы приводят к аварийному завершению программы с сообщением об ошибке.

Каждый из предопределенных классов `object`, `int`, `bool` и `str` определяет метод `__init__`. «Вызов» этих классов (при создании нового экземпляра объекта соответствующего типа) возвращает пустой объект, значение `0`, значение `False` и значение `""` (пустая строка) соответственно.

3. Лексическая структура

В этом разделе описываются детали, необходимые для реализации лексического анализатора для ChocoPy. Лексический анализ считывает входную программу и создает последовательности токенов. Токены

формируются из входной строки с использованием лексических правил, которые выражаются с помощью регулярных выражений. В случае неоднозначности токен содержит максимально длинную возможную строку, которая образует допустимый токен при чтении слева направо. Существуют следующие категории токенов: *структура строки*, *идентификаторы*, *ключевые слова*, *литералы*, *операторы* и *разделители*.

3.1. Структура строки

В ChocoPy, как и в Python, пробел может использоваться как для завершения оператора, так и для определения уровня отступа программного оператора. Чтобы учесть это, ChocoPy определяет три лексических токена, которые выводятся из пробела: `NEWLINE`, `INDENT` и `DEDENT`. Правила, когда такие токены генерируются, описаны далее с использованием концепций *физических* и *логических строк*.

3.1.1. Физические строки

Физическая строка — это последовательность символов, завершающаяся символом конца строки. В исходных файлах и строках могут использоваться следующие последовательности завершения строки: форма Unix с использованием ASCII LF (`\n`), форма Windows с использованием последовательности ASCII CR LF (`\r\n`) или старая форма Macintosh с использованием символа ASCII CR (`\r`). Все эти формы могут использоваться одинаково, независимо от платформы. Конец ввода также служит неявным терминатором для конечной физической строки.

3.1.2. Логические строки

Логическая строка — это физическая строка, которая содержит, по крайней мере, один токен, который не является пробелом или комментарием. Конец логической строки представлен лексическим токеном `NEWLINE`. Операторы не могут пересекать границы логических строк, за исключением случаев, когда `NEWLINE` допускается синтаксисом (например, между операторами в конструкциях формирования потока управления, таких как циклы `while`).

3.1.3. Комментарии

Комментарий начинается с символа решетки (`#`), который не является частью строкового литерала, и заканчивается в конце физической строки. Комментарии игнорируются лексическим анализатором; они не выдаются как токены.

3.1.4. Пустые строки

Физическая строка, которая содержит только пробелы, табуляции и, возможно, комментарий, игнорируется (т. е. токен `NEWLINE` не генерируется).

3.1.5. Отступы

Описание отступов заимствовано из документации Python 3[1].

«Начальные пробелы (пробелы и табуляции) в начале *логической строки* используются для вычисления уровня отступа строки, который, в свою очередь, используется для определения группировки операторов в блоки. Табуляции заменяются (слева направо) на от одного до восьми пробелов так, чтобы общее количество символов до заменяемого включительно было кратно восьми (это то же правило, которое используется в Unix). Общее количество пробелов, предшествующих первому непустому символу, затем определяет отступ строки».

«Уровни отступа последовательных строк используются для генерации токенов `INDENT` и `DEDENT` с использованием стека следующим образом: перед чтением первой строки входной программы в стек помещается один ноль; он больше никогда не будет извлечен. Числа, помещаемые в стек, всегда будут строго увеличиваться снизу вверх. В начале каждой логической строки уровень отступа строки сравнивается с верхом стека. Если он равен, ничего не происходит. Если он больше, он помещается в стек, и генерируется один токен `INDENT`. Если он меньше, это должно быть одно из чисел, находящихся в стеке; все числа в стеке, которые больше, извлекаются, и для каждого извлеченного числа генерируется токен `DEDENT`. В конце входной программы для каждого числа, оставшегося в стеке, которое больше нуля, генерируется токен `DEDENT`».

3.1.6. Пробелы между токенами

За исключением начала логической строки или строковых литералов, символы пробела и табуляции могут использоваться взаимозаменяемо для разделения токенов. Пробелы необходимы между двумя токенами только в том случае, если их конкатенация может быть интерпретирована как другой токен (например, ab — это один токен, но a b — это два токена). Символы пробела не являются токенами; они просто игнорируются.

---

[1] https://docs.python.org/3/reference/lexical_analysis.html

3.2. Идентификаторы

Идентификаторы определяются как непрерывная последовательность символов, содержащая заглавные и строчные буквы от A до Z, подчеркивание и цифры от 0 до 9 за исключением первого символа.

3.3. Ключевые слова

Следующие строки распознаются не как идентификаторы, а как отдельные ключевые токены: `False`, `None`, `True`, `and`, `as`, `assert`, `async`, `await`, `break`, `class`, `continue`, `def`, `del`, `elif`, `else`, `except`, `finally`, `for`, `from`, `global`, `if`, `import`, `in`, `is`, `lambda`, `nonlocal`, `not`,`or`, `pass`, `raise`, `return`, `try`, `while`, `with`, `yield`.

Не все ключевые слова имеют особое значение в ChocoPy. Например, ChocoPy не поддерживает `async` или `await`. Однако ChocoPy использует тот же список ключевых слов, что и Python, чтобы избежать случаев, когда идентификатор является допустимым в ChocoPy, но не в Python. Следовательно, некоторые ключевые слова (например, `async`) не встречаются нигде в грамматике и просто приведут к синтаксической ошибке. Идентификатор может содержать ключевое слово в качестве подстроки; например, `classic` является допустимым идентификатором, несмотря на то, что он содержит подстроку `class`. Это следует из правила самого длинного соответствия.

3.4. Литералы

Строковые и целочисленные литералы сопоставляются на этапе лексического анализа и представляются строковыми и целочисленными токенами соответственно. Структура этих литералов описана ниже. Булевы литералы `True` и `False` представляются просто своими ключевыми токенами.

3.4.1. Строковые литералы

Строковые литералы в ChocoPy значительно упрощены по сравнению с Python. В ChocoPy строковые литералы представляют собой просто последовательность символов ASCII, разделенных двойными кавычками (включая их): `" ... "`. Символы ASCII должны находиться в десятичном диапазоне `32-126` включительно, то есть быть больше или равными символу пробела и до тильды (~). Сама строка может содержать двойные кавычки, экранированные предшествующей обратной косой чертой, например \". Поскольку строковые литералы используются как для значений, так и для имен типов, удобно различать две категории: строковые литералы, содержимое которых имеет синтаксис идентификатора (`IDSTRING` в

синтаксисе), и другие строковые литералы (обозначаемые `STRING`). Значением строкового токена является последовательность символов между разделяющими двойными кавычками с использованием управляющих последовательностей (escape-последовательностей). Распознаются следующие управляющие последовательности: `\"`, `\n`, `\t`, `\\`, которые соответствуют двойной кавычке, новой строке, табуляции и обратной косой черте соответственно. Любая другая управляющая последовательность считается недопустимой. Вот несколько примеров:

Литерал	**Значение**
`"Hello"`	`Hello`
`"He\"ll\"o"`	`He"ll"o`
`"He\\\"llo"`	`He\"llo`
`"Hell\o"`	(ошибка: `"\o"` не распознано)

3.4.2. Целочисленные литералы

Целочисленные литералы в ChocoPy состоят из последовательности цифр от `0` до `9`, где самая левая цифра может быть `0`, только если это единственный символ в последовательности. То есть, ненулевые целочисленные литералы не могут иметь начальных нулей. Целочисленное значение таких литералов интерпретируется в десятичной системе счисления. Максимальное интерпретируемое значение может быть $2^{31} - 1$ для литерала `2147483647`. Литерал со значением, большим этого предела, приводит к лексической ошибке.

3.5.Операторы и разделители

Ниже приведен список символов, разделенных пробелами, которые соответствуют определенным токенам ChocoPy: `+ - * // % < > ≤ ≥ == ≠ = ( ) [ ] , : . ->`

4. Грамматика языка

На рисунке 3 представлена грамматика ChocoPy с использованием расширенной нотации BNF. Ключевые слова выделены жирным шрифтом. Литералы и пробельные символы представлены в ВЕРХНЕМ РЕГИСТРЕ. Нетерминалы отформатированы строчными курсивными буквами. Операторы и разделители отформатированы как есть. Нотация ⟦ ⟧ используется для группировки одного или нескольких символов в правиле грамматики. ⟦ и ⟧ не являются токенами входного языка. Символы или группы символов могут быть аннотированы следующим образом: «**?**» обозначает, что предыдущий

символ или группа являются необязательными, «`*`» обозначает ноль или более повторяющихся вхождений, а «`+`» обозначает одно или более повторяющихся вхождений. Разделение выражений на `expr` и `cexpr` отражает то, что ChocoPy (как и Python) допускает только логические бинарные или унарные выражения в качестве операндов логических операторов (`and`, `or`, `not`). В результате выражение `True == not False` должно вызывать синтаксическую ошибку (правильным выражением будет `True == (not False)`).

4.1. Приоритет и ассоциативность операторов

Операторы в ChocoPy имеют тот же приоритет, что и в Python. В следующей таблице приведен приоритет операторов в ChocoPy от самого низкого приоритета (наименьшая привязка) до самого высокого приоритета (наибольшая привязка).

Приоритет	Оператор(ы)	Ассоциативность
1	`· if · else ·`	Right
2	`or`	Left
3	`and`	Left
4	`not`	N/A
5	`==, ≠, <, >, ≤, ≥, is`	None
6	`+, - (binary)`	Left
7	`*, //, %`	Left
8	`- (unary)`	N/A
9	`., []`	Left

Обратите внимание, что операторы сравнения неассоциативны (поэтому ChocoPy, в отличие от Python 3, не допускает такие выражения, как `x < y < z`).

$$
\begin{aligned}
\textit{program} &::= [\![\textit{var_def} \mid \textit{func_def} \mid \textit{class_def}]\!]^{*}\ \textit{stmt}^{*} \\
\textit{class_def} &::= \textbf{class}\ \text{ID}\ (\ \text{ID}\ )\ :\ \text{NEWLINE INDENT}\ \textit{class_body}\ \text{DEDENT} \\
\textit{class_body} &::= \text{pass NEWLINE} \\
&\mid\ [\![\textit{var_def} \mid \textit{func_def}]\!]^{+} \\
\textit{func_def} &::= \textbf{def}\ \text{ID}\ (\ [\![\textit{typed_var}\ [\![,\ \textit{typed_var}]\!]^{*}]\!]^{?}\ )\ [\![\texttt{->}\ \text{type}]\!]^{?}\ :\ \text{NEWLINE INDENT}\ \textit{func_body}\ \text{DEDENT} \\
\textit{func_body} &::= [\![\textit{global_decl} \mid \textit{nonlocal_decl} \mid \textit{var_def} \mid \textit{func_def}]\!]^{*}\ \textit{stmt}^{+} \\
\textit{typed_var} &::= \text{ID}\ :\ \textit{type} \\
\textit{type} &::= \text{ID} \mid \text{IDSTRING} \mid [\ \textit{type}\ ] \\
\textit{global_decl} &::= \textbf{global}\ \text{ID NEWLINE} \\
\textit{nonlocal_decl} &::= \textbf{nonlocal}\ \text{ID NEWLINE} \\
\textit{var_def} &::= \textit{typed_var}\ \texttt{=}\ \textit{literal}\ \text{NEWLINE} \\
\textit{stmt} &::= \textit{simple_stmt}\ \text{NEWLINE} \\
&\mid\ \textbf{if}\ \textit{expr}\ :\ \textit{block}\ [\![\textbf{elif}\ \textit{expr}\ :\ \textit{block}\ ]\!]^{*}\ [\![\textbf{else}\ :\ \textit{block}]\!]^{?} \\
&\mid\ \textbf{while}\ \textit{expr}\ :\ \textit{block} \\
&\mid\ \textbf{for}\ \text{ID}\ \textbf{in}\ \textit{expr}\ :\ \textit{block} \\
\textit{simple_stmt} &::= \textbf{pass} \\
&\mid\ \textit{expr} \\
&\mid\ \textbf{return}\ [\![\textit{expr}]\!]^{?} \\
&\mid\ [\![\ \textit{target}\ \texttt{=}\ ]\!]^{+}\ \textit{expr} \\
\textit{block} &::= \text{NEWLINE INDENT}\ \textit{stmt}^{+}\ \text{DEDENT} \\
\textit{literal} &::= \textbf{None} \\
&\mid\ \textbf{True} \\
&\mid\ \textbf{False} \\
&\mid\ \text{INTEGER} \\
&\mid\ \text{IDSTRING} \mid \text{STRING} \\
\textit{expr} &::= \textit{cexpr} \\
&\mid\ \textbf{not}\ \textit{expr} \\
&\mid\ \textit{expr}\ [\![\textbf{and} \mid \textbf{or}]\!]\ \textit{expr} \\
&\mid\ \textit{expr}\ \textbf{if}\ \textit{expr}\ \textbf{else}\ \textit{expr} \\
\textit{cexpr} &::= \text{ID} \\
&\mid\ \textit{literal} \\
&\mid\ [\ [\![\textit{expr}\ [\![,\ \textit{expr}]\!]^{*}]\!]^{?}\ ] \\
&\mid\ (\ \textit{expr}\ ) \\
&\mid\ \textit{member_expr} \\
&\mid\ \textit{index_expr} \\
&\mid\ \textit{member_expr}\ (\ [\![\textit{expr}\ [\![,\ \textit{expr}]\!]^{*}]\!]^{?}\ ) \\
&\mid\ \text{ID}\ (\ [\![\textit{expr}\ [\![,\ \textit{expr}]\!]^{*}]\!]^{?}\ ) \\
&\mid\ \textit{cexpr}\ \textit{bin_op}\ \textit{cexpr} \\
&\mid\ \texttt{-}\ \textit{cexpr} \\
\textit{bin_op} &::= \texttt{+} \mid \texttt{-} \mid \texttt{*} \mid \texttt{//} \mid \texttt{\%} \mid \texttt{==} \mid \texttt{!=} \mid \texttt{<=} \mid \texttt{>=} \mid \texttt{<} \mid \texttt{>} \mid \textbf{is} \\
\textit{member_expr} &::= \textit{cexpr}\ .\ \text{ID} \\
\textit{index_expr} &::= \textit{cexpr}\ [\ \textit{expr}\ ] \\
\textit{target} &::= \text{ID} \\
&\mid\ \textit{member_expr} \\
&\mid\ \textit{index_expr}
\end{aligned}
$$

Рисунок 3. Грамматика языка ChocoPy.

5. Правила проверки типов

В этом разделе формально определяются правила проверки типов ChocoPy. Правила проверки типов определяют тип каждого выражения ChocoPy в заданном контексте. Контекст — это среда (environment) типов, которая описывает тип каждого несвязанного идентификатора, появляющегося в выражении. Среда типов описана в разделе 5.1. Раздел 5.2 содержит правила проверки типов.

5.1. Среды типов

Интуитивно проверку типов в ChocoPy можно рассматривать как восходящий алгоритм: тип выражения *e* выводится из (ранее выведенных) типов подвыражения e. Например, целое число 1 имеет тип int; в этом случае подвыражений нет. В качестве другого примера, если типы e1 и e2 — int, то выражение e1 > e2 имеет тип bool.

Сложность возникает в случае выражения v, где v — переменная или функция. Невозможно сказать, какой тип v в строго восходящем алгоритме; нам нужно знать тип, объявленный для v в большем выражении. Такое объявление должно существовать для каждой переменной и функции в допустимых программах ChocoPy. Чтобы получить информацию о типах идентификаторов, мы используем *среду типов*. *Среда* (environment) *типов* представляет собой четверку `<O, M, C, R>`: `O` — локальное окружение, `M` — окружение метода/атрибута, `C` — имя текущего класса, в котором появляется выражение или оператор, и `R` — возвращаемый тип функции или метода, в котором появляется выражение или оператор. `C` — принимает специальное значение ⊥, в случаях, когда выражение или оператор появляется вне класса, т. е. как оператор или выражение в глобальной области видимости. Аналогично, `R` — ⊥, в случаях, когда выражение или оператор появляется вне функции или метода, т. е. как оператор или выражение в глобальной области видимости программы. Локальное окружение и окружение метода/атрибута являются отображениями. Локальное окружение — это функция вида:

$$O(v) = T$$

которая назначает тип `T` переменной `v`. Это же окружение также содержит информацию о сигнатурах функций. Например,

$$O(f) = \{T_1 \times \cdots \times T_n \to T_0;\ x_1, \ldots, x_n;\ v_1 : T'_1, \ldots, v_m : T'_m\}$$

определяет тип f и указывает, что идентификатор f имеет формальные параметры $x_1, \ldots, x_n$ типов $T_1, \ldots, T_n$ соответственно и имеет возвращаемый тип T_0. Идентификаторы $v_1, \ldots, v_m$ являются переменными и вложенными функциями, объявленными в теле f, а их типы — $T'_1, \ldots, T'_m$ соответственно. Окружение Метода/атрибута аналогичным образом отображает класс и его атрибуты и методы в их типы. Например,

$$M(C, a) = T$$

отображает атрибут a в классе C в его тип T. Аналогично,

$$M(C, m) = \{T_1 \times \cdots \times T_n \to T_0;\ x_1, \ldots, x_n;\ v_1 : T'_1, \ldots, v_k : T'_k\}$$

отображает метод m класса C в его тип. В частности, он обозначает, что метод m в классе C имеет формальные параметры $x_1, \ldots, x_n$ типов $T_1, \ldots, T_n$ соответственно и имеет возвращаемый тип T_0. Идентификаторы $v_1, \ldots, v_k$ являются переменными и вложенными функциями, объявленными в теле m, а их типы — T соответственно.

Третий компонент среды типа — имя класса C, содержащего выражение или оператор для проверки типа. Четвертый компонент среды типа — возвращаемый тип R функции или метода, содержащего выражение или оператор для проверки типа.

При проверке типов определений функций и методов необходимо распространить среду типизации из внешней области действия в область действия функции, где наследуется привязка любого идентификатора, если только функция не объявляет формальный параметр, переменную или вложенную функцию с тем же именем. Пусть O — текущая локальное окружение, а f — функция с определением типа

$$\{T_1 \times \cdots \times T_n \to T_0;\ x_1, \ldots, x_n;\ v_1 : T'_1, \ldots, v_m : T'_m\}.$$

При проверке типов определения f мы проверяем тело с помощью локального окружения

$$O[T_1/x_1][T_2/x_2] \ldots [T_n/x_n][T'_1/v_1] \ldots [T'_m/v_m],$$

где обозначение O[T/c] используется для построения нового отображения следующим образом:

$$O[T/c](c) = T$$
$$O[T/c](d) = O(d) \text{ if } d \neq c$$

5.2. Правила проверки типов

Общая форма правила проверки типа имеет вид:

$$\frac{\vdots}{O, M, C, R \vdash e : T}$$

Правило следует читать так: в среде типа с локальным окружением `O`, окружением метода/атрибута `M`, содержащим класс `C` и возвращаемым типом `R`, выражение `e` имеет тип `T`. Запись под чертой является суждением о типе: символ турникета «⊢» отделяет контекст `(O, M, C, R)` от предложения `e : T`. Точки над горизонтальной чертой обозначают другие суждения о типах подвыражений `e`. Эти другие суждения являются гипотезами правила; если гипотезы удовлетворены, то суждение под чертой истинно.

Переменные. Правило для переменных простое: если в окружении идентификатору `id` присваивается тип `T`, то выражение `id` имеет тип `T`.

$$\frac{O(id) = T, \text{where } T \text{ is not a function type.}}{O, M, C, R \vdash id : T} \quad [\text{VAR-READ}]$$

Однако мы должны запретить идентификаторы с типами функций при чтении значений (то есть, когда идентификаторы используются как выражения в синтаксисе). Это просто отражает тот факт, что ChocoPy не рассматривает функции как объекты первого класса (присваиваемые, сохраняемые).

Определения переменных и присваивание значений. Правило для присваивания, как и другие, использует отношение $\leq_a$ (см. Раздел 2.4). Правило гласит, что назначенное выражение e_1 должно иметь тип T_1, совместимый по присваиванию с типом `T` идентификатора `id` в окружении типа.

$$\frac{\begin{array}{l} O(id) = T \\ O, M, C, R \vdash e_1 : T_1 \\ T_1 \leq_a T \end{array}}{O, M, C, R \vdash id = e_1} \quad [\text{VAR-ASSIGN-STMT}]$$

Для определения переменных использует похожее правило:

$$\frac{\begin{array}{l} O(id) = T \\ O, M, C, R \vdash e_1 : T_1 \\ T_1 \leq_a T \end{array}}{O, M, C, R \vdash id\colon T = e_1} \quad [\text{VAR-INIT}]$$

Двоеточие, используемое под линией в правиле для `VAR-INIT`, — это двоеточие в синтаксисе для аннотаций типов.

Последовательность определений и операторов. Данное правило проверяет все ли определения и операторы программы проходят проверку типов.

$$\frac{\begin{array}{l} O, M, C, R \vdash s_1 \\ O, M, C, R \vdash s_2 \\ \vdots \\ O, M, C, R \vdash s_n \\ n \geq 1 \end{array}}{O, M, C, R \vdash s_1 \text{ NEWLINE } s_2 \text{ NEWLINE } \ldots\ s_n \text{ NEWLINE}} \quad [\text{STMT-DEF-LIST}]$$

Оператор pass.

$$\frac{}{O, M, C, R \vdash \texttt{pass}} \quad [\text{PASS}]$$

Оператор выражений.

$$\frac{O, M, C, R \vdash e : T}{O, M, C, R \vdash e} \quad [\text{EXPR-STMT}]$$

Литералы.

$$\frac{}{O, M, C, R \vdash \texttt{False} : bool} \quad [\text{BOOL-FALSE}]$$

$$\frac{}{O, M, C, R \vdash \texttt{True} : bool} \quad [\text{BOOL-TRUE}]$$

$$\frac{i \text{ is an integer literal}}{O, M, C, R \vdash i : int} \quad [\text{INT}]$$

$$\frac{s \text{ is a string literal}}{O, M, C, R \vdash s : str} \quad [\text{STR}]$$

Литералу `None` присваивается специальный тип `<None>`:

$$\frac{}{O, M, C, R \vdash \texttt{None} : \texttt{<None>}} \quad [\text{NONE}]$$

Арифметические операции и операции сравнения над целыми числами.

$$\frac{O, M, C, R \vdash e : int}{O, M, C, R \vdash \texttt{-}\, e : int} \quad [\text{NEGATE}]$$

$$\frac{\begin{array}{c} O, M, C, R \vdash e_1 : int \\ O, M, C, R \vdash e_2 : int \\ op \in \{+, -, *, //, \%\} \end{array}}{O, M, C, R \vdash e_1\, op\ e_2 : int} \quad [\text{ARITH}]$$

$$\frac{\begin{array}{c} O, M, C, R \vdash e_1 : int \\ O, M, C, R \vdash e_2 : int \\ \bowtie \in \{\texttt{<}, \texttt{<=}, \texttt{>}, \texttt{>=}, \texttt{==}, \texttt{!=}\} \end{array}}{O, M, C, R \vdash e_1 \bowtie e_2 : bool} \quad [\text{INT-COMPARE}]$$

Логические операции.

$$\frac{\begin{array}{c} O, M, C, R \vdash e_1 : bool \\ O, M, C, R \vdash e_2 : bool \\ \bowtie \in \{\texttt{==}, \texttt{!=}\} \end{array}}{O, M, C, R \vdash e_1 \bowtie e_2 : bool} \quad [\text{BOOL-COMPARE}]$$

$$\frac{\begin{array}{c} O, M, C, R \vdash e_1 : bool \\ O, M, C, R \vdash e_2 : bool \end{array}}{O, M, C, R \vdash e_1\ \texttt{and}\ e_2 : bool} \quad [\text{AND}]$$

$$\frac{\begin{array}{c} O, M, C, R \vdash e_1 : bool \\ O, M, C, R \vdash e_2 : bool \end{array}}{O, M, C, R \vdash e_1\ \texttt{or}\ e_2 : bool} \quad [\text{OR}]$$

$$\frac{O, M, C, R \vdash e : bool}{O, M, C, R \vdash \texttt{not}\ e : bool} \quad [\text{NOT}]$$

Тернарное выражение (Выражение с условием).

$$\frac{\begin{array}{c} O, M, C, R \vdash e_0 : bool \\ O, M, C, R \vdash e_1 : T_1 \\ O, M, C, R \vdash e_2 : T_2 \end{array}}{O, M, C, R \vdash e_1 \texttt{ if } e_0 \texttt{ else } e_2 : T_1 \sqcup T_2)} \quad [\text{COND}]$$

Операции над строками.

$$\frac{\begin{array}{c} O, M, C, R \vdash e_1 : str \\ O, M, C, R \vdash e_2 : str \\ \bowtie \in \{\texttt{==}, \texttt{!=}\} \end{array}}{O, M, C, R \vdash e_1 \bowtie e_2 : bool} \quad [\text{STR-COMPARE}]$$

$$\frac{\begin{array}{c} O, M, C, R \vdash e_1 : str \\ O, M, C, R \vdash e_2 : str \end{array}}{O, M, C, R \vdash e_1 \texttt{ + } e_2 : str} \quad [\text{STR-CONCAT}]$$

$$\frac{\begin{array}{c} O, M, C, R \vdash e_1 : str \\ O, M, C, R \vdash e_2 : int \end{array}}{O, M, C, R \vdash e_1[e_2] : str} \quad [\text{STR-SELECT}]$$

Оператор `is`.

$$\frac{\begin{array}{c} O, M, C, R \vdash e_1 : T_1 \\ O, M, C, R \vdash e_2 : T_2 \\ T_1, T_2 \text{ are not one of } int, str, bool \end{array}}{O, M, C, R \vdash e_1 \texttt{ is } e_2 : bool} \quad [\text{IS}]$$

Конструктор объектов. Если `T` — имя класса, то выражения создания экземпляров объектов этого класса могут быть типизированы следующим образом:

$$\frac{T \text{ is a class}}{O, M, C, R \vdash T() : T} \quad [\text{NEW}]$$

Списки.

$$\frac{\begin{array}{l} n \geq 1 \\ O, M, C, R \vdash e_1 : T_1 \\ O, M, C, R \vdash e_2 : T_2 \\ \vdots \\ O, M, C, R \vdash e_n : T_n \\ T = T_1 \sqcup T_2 \sqcup \ldots \sqcup T_n \end{array}}{O, M, C, R \vdash \texttt{[}e_1, e_2, \ldots, e_n\texttt{]} : [T]} \quad [\text{LIST-DISPLAY}]$$

Правило для проверки типов пустого списка (особый случай):

$$\frac{}{O, M, C, R \vdash \texttt{[]} : \texttt{<Empty>}} \quad [\text{NIL}]$$

Операции над списками.

$$\frac{\begin{array}{l} O, M, C, R \vdash e_1 : [T_1] \\ O, M, C, R \vdash e_2 : [T_2] \\ T = T_1 \sqcup T_2 \end{array}}{O, M, C, R \vdash e_1 \texttt{+} e_2 : [T]} \quad [\text{LIST-CONCAT}]$$

$$\frac{\begin{array}{l} O, M, C, R \vdash e_1 : [T] \\ O, M, C, R \vdash e_2 : int \end{array}}{O, M, C, R \vdash e_1[e_2] : T} \quad [\text{LIST-SELECT}]$$

$$\frac{\begin{array}{l} O, M, C, R \vdash e_1 : [T_1] \\ O, M, C, R \vdash e_2 : int \\ O, M, C, R \vdash e_3 : T_3 \\ T_3 \leq_a T_1 \end{array}}{O, M, C, R \vdash e_1[e_2] = e_3} \quad [\text{LIST-ASSIGN-STMT}]$$

Доступ к атрибутам, присваивание значений атрибутам и инициализация атрибутов. Для доступа к атрибутам мы используем окружение члена класса `M`:

$$\frac{\begin{array}{l} O, M, C, R \vdash e_0 : T_0 \\ M(T_0, id) = T \end{array}}{O, M, C, R \vdash e_0.id : T} \quad [\text{ATTR-READ}]$$

$$\frac{\begin{array}{l} O, M, C, R \vdash e_0 : T_0 \\ O, M, C, R \vdash e_1 : T_1 \\ M(T_0, id) = T \\ T_1 \leq_a T \end{array}}{O, M, C, R \vdash e_0.id = e_1} \quad [\text{ATTR-ASSIGN-STMT}]$$

$$\frac{\begin{array}{l} M(C, id) = T \\ O, M, C, R \vdash e_1 : T_1 \\ T_1 \leq_a T \end{array}}{O, M, C, R \vdash id\colon T = e_1} \quad [\text{ATTR-INIT}]$$

Множественные присваивания. Множественное присваивание проверяется путем разложения на отдельные одиночные присваивания следующим образом.

$$\frac{\begin{array}{l} n > 1 \\ O, M, C, R \vdash e_0 : T_0 \\ O, M, C, R \vdash e_1 = e_0 \\ \vdots \\ O, M, C, R \vdash e_n = e_0 \\ T_0 \neq \texttt{[<None>]} \end{array}}{O, M, C, R \vdash e_1 = e_2 = \cdots = e_n = e_0} \quad [\text{MULTI-ASSIGN-STMT}]$$

Ограничение $\texttt{T}_0 \neq \texttt{[<None>]}$ позволяет избежать тонкой проблемы безопасности типов. Небезопасно допускать существование двух различных представлений списка с разными типами элементов. Тип

`[<None>]` в контексте множественного присвоения приводит ошибке как в приведенном фрагменте:

```
x: [A] = None
y: [[int]] = None
x = y = [None] # Trouble ahead!
x[0] = A()
print(y[0][0]) # ???
```

Java, например, попадает в эту ловушку и поэтому нуждается в `ArrayStoreException` времени выполнения. Чтобы избежать этого в ChocoPy, мы консервативно запрещаем присваивать значения типа `[<None>]` во множественном присваивании. Здесь есть еще один очень тонкий момент, таящийся в случае, когда e_0 имеет тип `<Empty>` (тип пустого списка). В этом случае нет необходимости использовать специальное правиле, потому что в ChocoPy нет метода `.append`, позволяющего добавлять элементы в пустой список. Если бы это было не так, мы могли бы получить следующую ситуацию:

```
A: [int] = None
B: [str] = None
A = B = []
A.append(3)
```

и впоследствии мы бы получили `B[0]`, возвращающее значение 3, которое, безусловно, не является строкой.

Вызов функций.

$$\frac{\begin{array}{l} O, M, C, R \vdash e_1 : T_1'' \\ O, M, C, R \vdash e_2 : T_2'' \\ \vdots \\ O, M, C, R \vdash e_n : T_n'' \\ n \geq 0 \\ O(f) = \{T_1 \times \cdots \times T_n \to T_0;\ x_1, \ldots, x_n;\ v_1 : T_1', \ldots, v_m : T_m'\} \\ \forall 1 \leq i \leq n : T_i'' \leq_a T_i \end{array}}{O, M, C, R \vdash f(e_1, e_2, \ldots, e_n) : T_0} \quad [\text{INVOKE}]$$

Для проверки типа вызова функции необходимо сначала проверить тип каждого аргумента функции. Тип каждого аргумента должен соответствовать

типам соответствующего формального параметра функции. Выражению вызова назначается объявленный тип возврата функции.

Проверка типа вызова метода выполняется аналогичным образом. Ключевое отличие от вызова функции заключается в том, что целевой метод определяется путем обращения к окружению метода/атрибута с использованием типа выражения объекта, метод которого вызывается:

$$\frac{\begin{array}{l} O, M, C, R \vdash e_1 : T_1'' \\ O, M, C, R \vdash e_2 : T_2'' \\ \vdots \\ O, M, C, R \vdash e_n : T_n'' \\ n \geq 1 \\ M(T_1'', f) = \{T_1 \times \cdots \times T_n \to T_0;\ x_1, \ldots, x_n;\ v_1 : T_1', \ldots, v_m : T_m'\} \\ T_1'' \leq_a T_1 \\ \forall i.\ 2 \leq i \leq n : T_i'' \leq_a T_i \end{array}}{O, M, C, R \vdash e_1.f(e_2, \ldots, e_n) : T_0} \quad [\text{DISPATCH}]$$

Операторы возврата. В этом правиле используется окружение возвращаемого типа значения:

$$\frac{\begin{array}{l} O, M, C, R \vdash e : T \\ T \leq_a R \end{array}}{O, M, C, R \vdash \texttt{return}\ e} \quad [\text{RETURN-E}]$$

$$\frac{\texttt{<None>} \leq_a R}{O, M, C, R \vdash \texttt{return}} \quad [\text{RETURN}]$$

Оператор условной передачи управления.

$$\frac{\begin{array}{l} O, M, C, R \vdash e_0 : bool \\ O, M, C, R \vdash b_0 \\ O, M, C, R \vdash e_1 : bool \\ O, M, C, R \vdash b_1 \\ \vdots \\ O, M, C, R \vdash e_n : bool \\ O, M, C, R \vdash b_n \\ n \geq 0 \\ O, M, C, R \vdash b_{n+1} \end{array}}{O, M, C, R \vdash \texttt{if}\ e_0\texttt{:}b_0\ \texttt{elif}\ e_1\texttt{:}b_1 \ldots\ \texttt{elif}\ e_n\texttt{:}b_n\ \texttt{else:}b_{n+1}} \quad [\text{IF-ELIF-ELSE}]$$

Оператор `while`.

$$\frac{\begin{array}{l} O, M, C, R \vdash e : bool \\ O, M, C, R \vdash b \end{array}}{O, M, C, R \vdash \texttt{while}\ e\texttt{:}b} \quad [\text{WHILE}]$$

Оператор for.

$$\frac{\begin{array}{l} O, M, C, R \vdash e : str \\ O(id) = T \\ str \leq_a T \\ O, M, C, R \vdash b \end{array}}{O, M, C, R \vdash \texttt{for}\ id\ \texttt{in}\ e\texttt{:}b} \quad [\text{FOR-STR}]$$

$$\frac{\begin{array}{l} O, M, C, R \vdash e : [T_1] \\ O(id) = T \\ T_1 \leq_a T \\ O, M, C, R \vdash b \end{array}}{O, M, C, R \vdash \texttt{for}\ id\ \texttt{in}\ e\texttt{:}b} \quad [\text{FOR-LIST}]$$

Определения функций. Чтобы проверить тип определения функции `f`, необходимо проверить тело функции `f` в окружении, где `O` расширено привязками для имен, явно объявленных функцией `f`.

$$T = \begin{cases} T_0, & \text{if } \texttt{->} \text{ is present,} \\ \texttt{<None>}, & \textit{otherwise.} \end{cases}$$

$$\frac{\begin{array}{l} O(f) = \{T_1 \times \cdots \times T_n \to T;\ x_1, \ldots, x_n;\ v_1 : T'_1, \ldots, v_m : T'_m\} \\ n \geq 0 \qquad m \geq 0 \\ O[T_1/x_1] \ldots [T_n/x_n][T'_1/v_1] \ldots [T'_m/v_m], M, C, T \vdash b \end{array}}{O, M, C, R \vdash \texttt{def}\ f(x_1 \texttt{:} T_1, \ldots, x_n \texttt{:} T_n)\ [\![\texttt{->}\ T_0]\!]^{?} \texttt{:} b} \quad [\text{FUNC-DEF}]$$

Аналогично для методов:

$$T = \begin{cases} T_0, & \text{if } \texttt{->} \text{ is present,} \\ \texttt{<None>}, & \textit{otherwise.} \end{cases}$$

$$\frac{\begin{array}{l} M(C, f) = \{T_1 \times \cdots \times T_n \to T;\ x_1, \ldots, x_n;\ v_1 : T'_1, \ldots, v_m : T'_m\} \\ n \geq 1 \qquad m \geq 0 \\ C = T_1 \\ O[T_1/x_1] \ldots [T_n/x_n][T'_1/v_1] \ldots [T'_m/v_m], M, C, T \vdash b \end{array}}{O, M, C, R \vdash \texttt{def}\ f(x_1 \texttt{:} T_1, \ldots, x_n \texttt{:} T_n)\ [\![\texttt{->}\ T_0]\!]^{?} \texttt{:} b} \quad [\text{METHOD-DEF}]$$

Определения классов. Проверка типов для определения классов выполняется путем распространения соответствующего окружения типов:

$$\frac{O, M, C, R \vdash b}{O, M, \bot, R \vdash \texttt{class}\ C(S) \texttt{:} b} \quad [\text{CLASS-DEF}]$$

Глобальная окружение типов. Следующие функции и методы классов предопределены глобально:

$$\begin{aligned} O(len) &= \{object \to int;\ arg\} \\ O(print) &= \{object \to \texttt{<None>};\ arg\} \\ O(input) &= \{\to str\} \\ M(object, __init__) &= \{object \to \texttt{<None>};\ self\} \\ M(str, __init__) &= \{object \to \texttt{<None>};\ self\} \\ M(int, __init__) &= \{object \to \texttt{<None>};\ self\} \\ M(bool, __init__) &= \{object \to \texttt{<None>};\ self\} \end{aligned}$$